

CINEEXPLORER

Rapport Final - Phase 4

Interface Web Django - Architecture Multi-Bases

5 Pages

Interface Web

2 Bases

SQLite + MongoDB

291K Films

11.8M Documents

Bootstrap 5

Design Responsive

Status du projet: COMPLET

Phase 1: SQLite	COMPLET	12 tables, 11.8M lignes
Phase 2: MongoDB	COMPLET	Documents structures
Phase 3: Replica Set	COMPLET	3 noeuds, failover teste
Phase 4: Django	COMPLET	5 pages, responsive

CinéExplorer - Projet BDA | Decembre 2025

Table des matieres

1. Architecture globale	3
2. Choix technologiques	4
3. Description des fonctionnalites	5
4. Strategie multi-bases	6
5. Benchmarks de performance	7
6. Difficultes et solutions	8
7. Conclusion	9
Annexes	10

1. Architecture globale

L'architecture de CineExplorer repose sur une approche multi-bases combinant SQLite pour les operations relationnelles et MongoDB en Replica Set pour les documents structures. Cette architecture permet d'exploiter les forces de chaque technologie tout en assurant haute disponibilite et tolerance aux pannes.

Schema d'architecture

COUCHE PRESENTATION
Django 6.0 + Bootstrap 5
Templates HTML Chart.js Responsive Design
COUCHE SERVICES
SQLiteService MongoDBService
Abstraction des acces aux donnees
COUCHE DONNEES
SQLite (imdb.db) MongoDB Replica Set (rs0)
786 MB 3 noeuds (27017, 27018, 27019)

Composants principaux

Composant	Role	Technologie
Frontend	Interface utilisateur responsive	Bootstrap 5, Chart.js
Backend	Logique metier, routing	Django 6.0, Python 3.12
Base relationnelle	Listes, filtros, recherche	SQLite 3
Base documents	Details films complets	MongoDB 7.0
Replica Set	Haute disponibilite	3 noeuds MongoDB

2. Choix technologiques

Stack technologique

Categorie	Technologie	Version	Justification
Framework Web	Django	6.0	ORM puissant, admin integre, securite
Base SQL	SQLite	3.x	Leger, sans serveur, performant
Base NoSQL	MongoDB	7.0	Documents flexibles, agregations
Frontend CSS	Bootstrap	5.3	Responsive, composants riches
Graphiques	Chart.js	4.x	Interactif, leger, personnalisable
Python	Python	3.12	Ecosysteme riche, typage

Pourquoi SQLite + MongoDB ?

La combinaison de SQLite et MongoDB permet d'exploiter le meilleur des deux mondes:

SQLite - Points forts:

- Requetes relationnelles complexes (JOINS, GROUP BY, Window functions)
- Filtrage et tri efficaces avec indexes B-tree
- Zero configuration, base embarquee
- Ideal pour les listes paginées et recherches

MongoDB - Points forts:

- Documents denormalises (1 requete = toutes les infos d'un film)
- Schema flexible pour donnees heterogenes
- Replica Set pour haute disponibilite
- Pipeline d'agregation puissant

3. Description des fonctionnalites

L'application CineExplorer propose 5 pages principales permettant d'explorer la base de donnees IMDB de maniere intuitive et performante.

Page	URL	Fonctionnalites	Source
Accueil	/	Stats, Top 10 films, Status bases	SQLite + MongoDB
Liste Films	/movies/	Pagination, Filtres, Tri	MongoDB
Detail Film	/movies/<id>/	Infos completes, Casting, Similaires	MongoDB
Recherche	/search/	Films et Personnes	MongoDB
Statistiques	/stats/	Graphiques, Distributions	SQLite + MongoDB

Page Liste des Films (/movies/)

Fonctionnalite	Implementation
Pagination	20 films par page, navigation complete
Filtre par genre	Liste deroulante dynamique (28 genres)
Filtre par annee	Plage min-max (1900-2025)
Filtre par note	Note minimale (5+ a 9+)
Tri	Titre, Annee, Note, Popularite (ASC/DESC)
Cards responsive	Grille adaptative 1-4 colonnes

Page Detail Film (/movies//)

Section	Contenu
En-tete	Titre, Annee, Duree, Note, Genres
Realisateurs	Liste avec avatars
Casting	Top 10 acteurs avec personnages
Scenaristes	Liste complete

Titres alternatifs	Par region (8 max)
Films similaires	6 films du meme genre

4. Strategie multi-bases

La strategie multi-bases consiste a utiliser chaque base de donnees pour ses points forts, optimisant ainsi les performances globales de l'application.

Repartition des responsabilites

Operation	Base utilisee	Justification
Liste films + filtres	MongoDB	Documents pre-agreges, 1 requete
Detail complet film	MongoDB	Toutes infos en 1 document
Recherche textuelle	MongoDB	Index text, regex
Statistiques globales	SQLite	Agregations SQL optimisees
Top acteurs/realisateurs	SQLite	JOINS multiples efficaces
Films par decennie	SQLite	GROUP BY performant

Implementation des Services

Deux services abstraient l'accès aux données:

Service	Methodes principales
SQLiteService	get_stats(), get_movies(), search_movies(), get_movies_by_genre_stats()
MongoDBService	get_movie_complete(), get_movies_complete(), get_similar_movies()

Avantages de cette approche

- Performances optimales: chaque base pour ses forces
- Flexibilite: facile d'ajouter de nouvelles sources
- Maintenabilite: separation claire des responsabilites
- Scalabilite: MongoDB Replica Set pour la HA

5. Benchmarks de performance

Les benchmarks ont ete realises avec 5 iterations par operation pour obtenir des moyennes fiables. Les tests comparent SQLite et MongoDB sur les memes operations.

Resultats comparatifs

Operation	SQLite (ms)	MongoDB (ms)	Gagnant	Ratio
Stats generales	126.29	275.64	SQLite	2.2x
Liste films (20)	166.85	0.57	MongoDB	292x
Liste filtree	118.44	0.72	MongoDB	164x
Detail film	0.34	1.40	SQLite	4.1x
Recherche	20.51	64.35	SQLite	3.1x
Stats par genre	424.76	296.97	MongoDB	1.4x
Films/decennie	120.53	299.28	SQLite	2.5x

Analyse des resultats

MongoDB excelle pour la liste des films (292x plus rapide) grace aux documents denormalises qui evitent les JOINS. SQLite reste superieur pour les requetes simples (detail film) et certaines agregations (films par decennie).

La strategie multi-bases est validee par les benchmarks

6. Difficultes et solutions

Le developpement de CineExplorer a presente plusieurs defis techniques qui ont ete resolus par des solutions adaptees.

Difficulte	Cause	Solution
Variables _id Django	Django interdit les variables commençant par _	Fonction fix_id() renomme _id en id
Page stats lente (10s)	Agregations sur 291K documents	Cache en memoire (5 min) + Index MongoDB
Genres non affiches	get_all_genres() silencieusement vide	Ajout logging + API debug /api/genres/
Replica Set election	Temps d'election variable	Configuration heartbeat optimisee
Pagination lourde	Chargement de tous les films	Limite a 1000 + pagination cote serveur
Filtres non persistants	Parametres perdus entre pages	Conservation dans URL query string

Solution detaillee: Probleme _id

Django n'autorise pas les variables commençant par underscore dans les templates. Or MongoDB utilise _id comme identifiant. La solution implementee:

```
def fix_id(doc):
    if doc and '_id' in doc:
        doc['id'] = doc.pop('_id')
    return doc
```

Solution detaillee: Optimisation Stats

La page statistiques chargeait en 10 secondes. Solutions appliquees:

- Cache en memoire avec TTL de 5 minutes
- Sampling MongoDB (\$sample) limite a 50K documents
- Index sur genres, year, rating.average, rating.votes
- Script create_indexes.py pour reproductibilite

7. Conclusion

Objectifs atteints

Objectif	Status	Details
Interface web professionnelle	COMPLET	5 pages Bootstrap 5 responsive
Integration multi-bases	COMPLET	SQLite + MongoDB optimises
Design responsive	COMPLET	Mobile-first, 4 breakpoints
Graphiques interactifs	COMPLET	Chart.js sur /stats/
Haute disponibilite	COMPLET	Replica Set 3 noeuds
Performance	COMPLET	< 1ms pour listes MongoDB

Apprentissages cles

- L'architecture multi-bases offre flexibilite et performance
- La denormalisation MongoDB accelere drastiquement les lectures
- Les index doivent etre cibles (pas de sur-indexation)
- Le cache est essentiel pour les agregations couteuses
- Django + Bootstrap permettent un developpement rapide

Perspectives d'evolution

- Ajout d'authentification utilisateur
- Systeme de favoris et watchlist
- API REST pour applications mobiles
- Recommandations basees sur l'historique
- Recherche full-text avec Elasticsearch

Annexes

A. Structure du projet

```
cineexplorer/  
  config/          # Configuration Django  
  movies/          # Application principale  
    services/      # SQLiteService, MongoDBService  
    templates/     # Templates HTML  
    views.py       # Vues Django  
  scripts/         # Scripts utilitaires  
    create_indexes.py  
    benchmark_phase4.py  
  data/            # Donnees  
    imdb.db        # Base SQLite  
    mongo/         # Donnees MongoDB  
  static/          # Fichiers statiques  
  reports/         # Rapports PDF
```

B. Commandes utiles

Commande	Description
python manage.py runserver	Demarrer le serveur Django
./scripts/phase3_replica/setup_replica.sh start	Demarrer le Replica Set
python scripts/create_indexes.py	Creer les index MongoDB
python scripts/benchmark_phase4.py	Executer les benchmarks

C. URLs de l'application

URL	Description
/	Page d'accueil
/movies/	Liste des films
/movies/<id>/	Detail d'un film
/search/	Recherche
/stats/	Statistiques
/api/test/	Test connexions (JSON)

/api/genres/	Liste des genres (JSON)
--------------	-------------------------